



Tecnológico de Costa Rica

Campus Central Cartago

Proyecto 1 - Gramatica

IC5701 - Compiladores e intérpretes

Profesor: Aurelio Sanabria

II Semestre - 2026

Estudiantes:

Isaac Jiménez Blanco - 2024202804

Andrés Gabriel Padilla Robles - 2025137295

Alina Rodriguez Wong - 2024063444

Neithan Vargas Vargas - 2025149384

Adolfo Barquero Monestel - 2022211388

24 de septiembre de 2026

1. Motivación.

◆ El lenguaje desarrollado, llamado NecroScript, es un lenguaje estructurado y de tipado estático. La sintaxis y palabras reservadas, giran en torno a la temática de la necromancia y su objetivo es presentar una gramática distintiva y clara para permitir al programador desarrollar algoritmos utilizando una metáfora consistente. El lenguaje cubre los elementos básicos de un lenguaje de programación de propósito general: ◆

- { Declaración de variables tipadas (calavera, cantico, esencia, latido, fosa, ceniza)}
- { Declaración de funciones llamadas rituales }
- { Estructuras de control (mana, sin mana, sin) }
- { Estructuras de repetición (convocar y persistir) }
- { Llamadas a funciones base (invocar, revivir) }
- { Expresiones aritmeticas y logicas }
- { Arreglos, comentarios y bloques delimitados por símbolos }

1.1. Selección de la temática

El lenguaje desarrollado por el grupo se basa en la temática de Magia y nigromancia. Cada elemento sintáctico del lenguaje (palabras reservadas, tipos, operadores y símbolos) fue adaptado a esta temática para darle personalidad propia al lenguaje. Esto da una gran versatilidad para desarrollar el lenguaje, pues se cuenta con un vocabulario amplio y reconocible, que permite adaptar los distintos elementos de los lenguajes de programación sin tener que utilizar traducciones forzadas del inglés. Además, la selección de esta temática permite el uso de simbolos específicos ({} , ◆ ◆, ⊕, ℄, ∞, ~) no solo de forma funcional, sino que estos refuerzan la metáfora con la que se desarrolla.

1.2. Adaptación del lenguaje a la temática:

La idea de este lenguaje no es solo sustituir palabras, sino que cada una de las estructuras como funciones, bloque o expresiones aritméticas y lógicas tuvieran una relación conceptual con la temática de la necromancia. La siguiente tabla muestra algunas de los elementos adaptados:

Palabras reservadas

Palabra reservada	Utilidad
ritual	Declara una función
sacrificar	Retorna un valor de una función (return)
mana	Estructura de bifurcación (if)
sin mana	Bifurcación alternativa (else if)
sin	Bifurcación por defecto (else)
convocar ... en ...	Bucle de repetición controlado (for)
persistir	Bucle de repetición condicional (while)
calavera	Tipo de dato entero (int)
cantico	Tipo de dato texto/cadena (string)
esencia	Tipo de dato flotante/decimal (float)
latido	Tipo de dato booleano (bool)
ceniza	Valor nulo/vacío (null / void)
fosa	Prefijo que indica que el tipo es una lista/arreglo
invocar	Función estándar de print
revivir	Función estándar de calloc
y / o	Operadores lógicos (and / or)
vivo / muerto	Boleanos (verdadero / falso)

Símbolos reservados

Símbolo	Uso
}	Inicio de argumentos y parámetros
}	Fin de argumentos y parámetros
◆	Inicio de bloque
◇	Fin de bloque
⊕	Fin de instrucción
<-	Asignación
∞	Asignación de tipo de retorno
~~	Comentarios
()	Brackets para arreglos/listas
&	Ampersand de referencia
ensamblar	Suma (+)
mutilar	Resta (-)
infestar	Multiplicación (*)
desmembrar	División (/)
restos	Residuo/Módulo (%)
==, !=, >, <, >=, <=	Operadores de comparación
,	Separador de parámetros, elementos. Es una , normal.

2. Elementos sólidos y chistosos del lenguaje.

2.1. Fortalezas

Uno de los puntos fuertes de Necroscrypt es que no se limita únicamente a reemplazar palabras, sino que construye un sistema diferente y único al emplear símbolos. Los delimitadores de los bloques ◆ ◇ , los de argumentos } } y los terminadores de instrucción

⊕ no tiene un equivalente en lenguajes como Python o Java, por lo que un programa escrito en Necroscript es fácilmente reconocible. Por otro lado, lenguajes como C++ comparten casi la misma sintaxis, por lo que escribir el código puede llegar a sentirse, hasta cierto punto, familiar para algunas personas.

Además de esto cada una de las palabras tiene una justificación conceptual dentro de la necromancia, lo que reduce la carga cognitiva de las personas al poder relacionar fácilmente los términos con el objetivo que cumplen. Esto contrasta con lenguajes como ASM, donde los mnemónicos utilizados no guardan una relación con su función que sea fácilmente reconocible. Finalmente, a pesar de que Necroscript es un lenguaje temático, cumple con todos los elementos mínimos de un lenguaje de programación de propósito general contando con tipos primitivos, funciones, arreglos, estructuras de control y expresiones aritméticas y lógicas que permiten el desarrollo de diversos algoritmos.

Desde el punto de vista de los compiladores, se puede mencionar que Necroscript presenta una modularidad alta, y flexibilidad alta, debido a la transcendencia a python, donde el lenguaje python es de alta modularidad, y flexibilidad entre sistemas operativos.

2.2. Debilidades y Aspectos Chistosos

En Necroscript, se presentan varias debilidades, debido a sus decisiones de diseño, la primera siendo uso obligatorio de UTF en vez de ASCII: debido al uso de caracteres especiales y símbolos como ♦, ✧, }, {, ⊕, ℄, ☺ y ∞, tener un archivo de ascii, o estar en terminales viejas, rompería el programa, no lo haría imposible programar en. Aparte que los archivos usaron una mayor cantidad de memoria, por ser utf-8. Estos son reemplazos de los caracteres normalmente utilizados en lenguajes populares, de estilo imperativo, o orientado a

objetos, como C, Rust, Javascript, donde se utilizan (), {}, estos serian una contraparte temática.

Para considerar el uso de python, también se tiene que tomar en cuenta el rendimiento, este lenguaje va a ser lento, tanto en la etapa de compilación como en la de ejecución. La lentitud la de compilación se debe a que el compilador (En realidad es un transpilador) estará programado en python, por la naturaleza del lenguaje python, este no va a tener un tiempo de compilación rápido. Por otro lado, la lentitud la de ejecución se debe al tener una comprobación para el mana, esto agrega complejidad a cada instrucción. Al final, el módulo de mana se agrega al archivo compilado final, en lenguaje python, por lo que si generara mucha complejidad en el tiempo de ejecución.

Otra cosa chistosa del programa, serían los errores tipo "Mana fault" durante la ejecución. Debido a que el maná no se puede evaluar totalmente en tiempo de compilación, por cosas como user inputs, y algoritmos no determinísticos, los Mana fault serán similares a los segmentation fault, un error, pero el programa brindará ayuda a nivel de frase, con la línea en la que se acabo el mana.

3. Gramática EBNF

Significado de los colores:

- **no-terminales (reglas)**
- **palabras reservadas**
- **símbolos del lenguaje**
- **símbolos EBNF**

Programa ::= Elemento | Elemento Programa

Elemento ::= Ritual | Instruccion

Ritual ::= "ritual"; Nombre; "}" ; Parametros; "{" ; TipoRetorno; Bloque

Parametros ::= Parametro | Parametro "," Parametros | ϵ

Parametro ::= Tipo; Nombre

TipoRetorno ::= ϵ | "○○"; Tipo

Bloque ::= "◆"; Instrucciones; "◇"

Instrucciones ::= Instruccion; Instrucciones | Instruccion | ϵ

Instruccion ::= Comentario | Declaracion; "⊕"; [Comentario] | Asignacion; "⊕";

[Comentario] | Llamada; "⊕"; [Comentario] | Sacrificio; "⊕"; [Comentario] | Condicional;

[Comentario] | Bucle; [Comentario] "n"

Comentario ::= "~~"; TextoComentario; "~~"

TextoComentario ::= CaracterComentario | CaracterComentario; TextoComentario | ϵ

CaracterComentario ::= LetraComentario | Numero | Simbolo

LetraComentario ::= [a-z] | [A-Z] | "á" | "é" | "í" | "ó" | "ú" | "Á" | "É" | "Í" | "Ó" | "Ú" | "ñ" | "Ñ"

Tipo ::= TipoPrimitivo | TipoArreglo

TipoPrimitivo ::= "calavera" | "cantico" | "esencia" | "latido" | "ceniza"

TipoLista ::= "calavera" | "cantico" | "esencia" | "latido"

TipoArreglo ::= "fosa"; TipoLista

Nombre ::= Letra; RestoNombre

RestoNombre ::= Letra | Numero | "_" | Letra; RestoNombre | Numero; RestoNombre | "_";
RestoNombre | ϵ

Declaracion ::= "calavera"; Nombre; "<"; ValorCalavera | "cantico"; Nombre; "<";
ValorCantico | "esencia"; Nombre; "<"; ValorEsencia | "latido"; Nombre; "<"; ValorLatido |
"ceniza"; Nombre; "<"; "ceniza" | "fosa"; "calavera"; Nombre; "<"; "⌈";
ListaValorCalavera; "⌋" | "fosa"; "cantico"; Nombre; "<"; "⌈"; ListaValorCantico; "⌋" |
"fosa"; "esencia"; Nombre; "<"; "⌈"; ListaValorEsencia; "⌋" | "fosa"; "latido"; Nombre;
"<"; "⌈"; ListaValorLatido; "⌋";

Asignacion ::= Nombre; "<"; Expresion

Sacrificio ::= "sacrificar"; Expresion

Condicional ::= "mana"; "⌋"; ExpresionLogica; "⌋"; Bloque; BloqueSinMana; BloqueSin
BloqueSinMana ::= "sin mana"; "⌋"; ExpresionLogica; "⌋"; Bloque; BloqueSinMana | ϵ
BloqueSin ::= "sin"; Bloque | ϵ

Bucle ::= BucleConvocar | BuclePersistir

BucleConvocar ::= "convocar"; Tipo; Nombre; "en"; Expresion; Bloque

BuclePersistir ::= "persistir"; "⌋"; ExpresionLogica; "⌋"; Bloque

ValorCalavera ::= Expresion

ValorCantico ::= Expresion

ValorEsencia ::= Expresion

ValorLatido ::= Booleano | ExpresionLogica

ListaValorCalavera ::= ValorCalavera | ValorCalavera; ","; ListaValorCalavera | ϵ

ListaValorCantico ::= ValorCantico | ValorCantico; ","; ListaValorCantico | ϵ

ListaValorEsencia ::= ValorEsencia | ValorEsencia; ","; ListaValorEsencia | ϵ

ListaValorLatido ::= ValorLatido | ValorLatido; ","; ListaValorLatido | ϵ

Llamada ::= LlamadaRitual

LlamadaRitual ::= Nombre; "{"; Argumentos; "}"

Argumentos ::= Argumento | Argumento; ","; Argumentos | ε

Argumento ::= Expresion | Tipo

ExpresionLogica ::= Comparacion | Comparacion; OperadorLogico; ExpresionLogica |
Booleano

Comparacion ::= Expresion; OperadorComparacion; Expresion

OperadorComparacion ::= "==" | "!=" | ">" | "<" | ">=" | "<=";

OperadorLogico ::= "y" | "o"

Expresion ::= Termino | Termino; OperadorAritmetico; Expresion

OperadorAritmetico ::= "ensamblar" | "mutilar" | "infestar" | "desmembrar" | "restos";

Termino ::= Decimal | Numero | Cadena | Booleano | "ceniza" | LlamadaRitual | Nombre |
"}"; Expresion; "}"

Cadena ::= ""; Texto; "";

Texto ::= CaracterTexto | CaracterTexto; Texto | ε

CaracterTexto ::= Letra | Numero | Simbolo

Numero ::= [0-9]+

Letra ::= [a-z] | [A-Z]

Simbolo ::= " " | "," | "." | "!" | "?" | ":" | ";" | "-" | "_" | "(" | ")" | "+" | "*" | "/" | "%" | "=" | "<" |
">" | "&" | "|" | "#" | "¿" | "¡"

Decimal ::= Numero; "."; Numero;

4. Ejemplos de código en su gramática. Al menos 5 ejercicios

Variables:

```
ritual main {  
  calavera a <- 20  
  calavera b <- 6  
  cantico nombre <- "Pepe el mago"  
  esencia pi <- 3.14  
}
```

Mana es como un if

```
ritual fibonacci { calavera n }  
  mana { n <= 1  
    sacrificar 1  
  }  
  sin  
    sacrificar fibonacci { n mutilar 1 } ensamblar fibonacci { n mutilar 2 }  
}
```

Uso de los tres condicionales

```
ritual nivel_de_mana { calavera mana_actual }  
  mana { mana_actual >= 100  
    invocar { "Mana lleno" }  
  }  
  sin mana { mana_actual >= 50  
    invocar { "Mana medio" }  
  }
```

```

sin ✦
  invocar{"Mana bajo"} ✦
✦
✦

```

Uso de bucle for

```

ritual imprimir_cuadrado{calavera n} ✦
  convocar calavera i en mundo{n} ✦
    convocar calavera j en mundo{n} ✦
      invocar_sin_salto{"#"} ✦ ~~ Función estándar programada por nosotros para
imprimir sin salto de línea ~~
    ✦
  invocar{""} ✦
✦
✦

```

Uso del while

```

ritual invocar_hechizo{calavera mana_actual} ✦
  persistir{mana_actual > 0} ✦
    invocar{"El hechizo sigue activo"} ✦
    mana_actual <- mana_actual mutilar 10 ✦
  ✦
✦

```

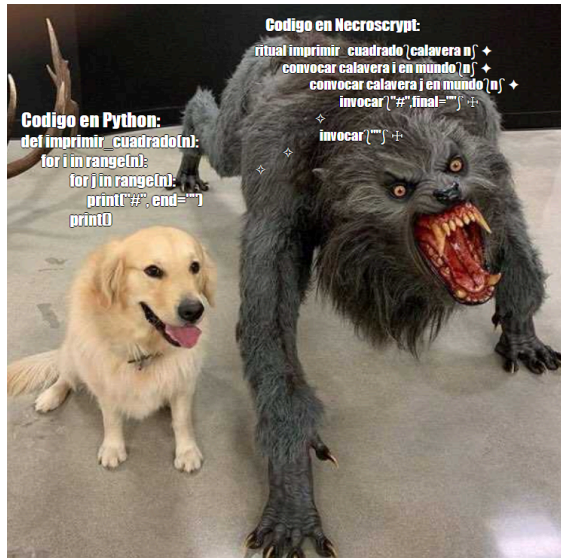
Uso de listas

```
ritual manejo_fosa { } ✦  
  fosa calavera almas <- ( 10, 20, 30 ) ✦  
  
  invocar { almas } ✦  
  
  insertar { almas, 40 } ✦  
  
  calavera posicion <- buscar { almas, 20 } ✦  
  invocar { posicion } ✦  
  
  calavera extraida <- extraer { almas, 1 } ✦  
  invocar { extraida } ✦  
  
  invocar { almas } ✦
```

✦

5. Extras

5.1 Memes

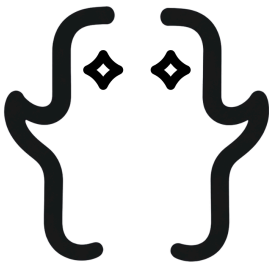


Yo recibiendo 500 líneas de error
del analizador sintáctico porque
me faltó poner un “}”

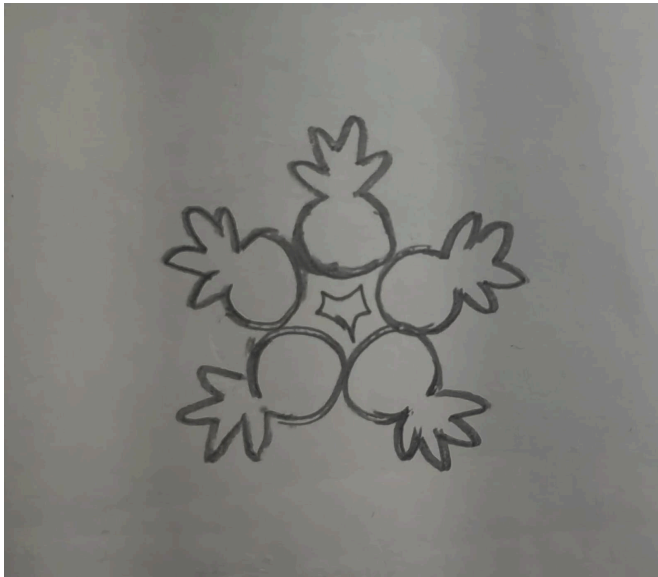


5.2 Logos

Logo de Necroscript



Logo del equipo



5.3. Foto icónica



5.4 Audiovisual

5.4.1 Se encuentra un el video adjunto en el repositorio.

5.4.2 Se encuentra la Infografía:

